An Efficient Certificateless Signature Scheme With Provably Security and Its Applications

Kui Ma, Yanwei Zhou, Ying Wang, Chunsheng Dong, Zhe Xia, Bo Yang, and Mingwu Zhang

***Abstract*—The Internet of Things (IoT) is helpful in making people's life more convenient and efficient. To ensure that the nodes within IoT can interact securely, a certificateless signature (CLS) can be used to protect message authentication in the IoT. Recently, some concrete constructions of CLS schemes have been proposed in the literature, but through our analyses, we demonstrate that some existing CLS schemes cannot keep their claimed security because of various security flaws. For example, a valid signature can be forged by any Type I adversary by replacing the corresponding user's public key. In this article, we describe the security flaws that need to be addressed and introduce a novel CLS scheme without using bilinear mapping. The existential unforgeability in our proposed scheme can be proved based on the hardness of the elliptic curve discrete logarithm problem in the random oracle model. The comparison with existing CLS schemes shows that our scheme not only enjoys more security features but also is more efficient in computation. Moreover, we introduce an identity authentication protocol as the application of our proposed CLS scheme, which achieves mutual authentication and anonymity in communications.**

***Index Terms*—Certificateless public key cryptography, certificateless signature (CLS), Internet of Things (IoT), provably security.**

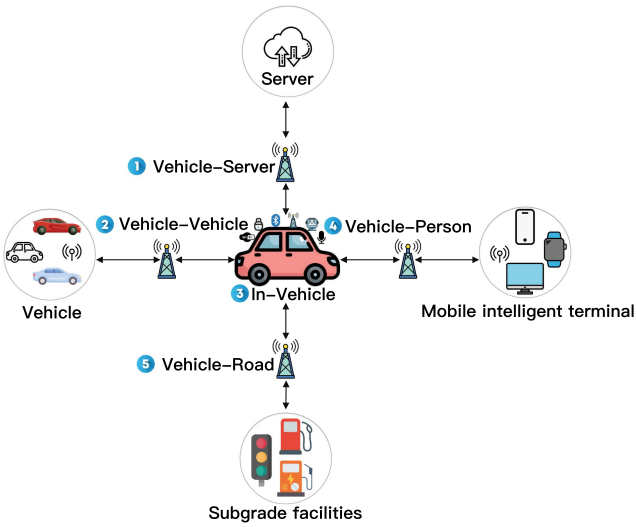

Fig. 1. Composition of vehicle ad hoc network.

I. Introduction

IN THE Internet of Things (IoT), everything can be connected for information exchange. Therefore, it can be used in many fields, such as vehicular ad hoc networks (VANET), smart homes, smart factories, etc. [1], [2]. However, with the rapid increase in the number of devices connected to IoT, the system framework is becoming more and more complex, and security problems are raising. For example, an attacker can tamper with messages during communication in various ways. The consequence is that messages received by the user may be inconsistent with the original ones, or the receiver cannot verify the source of the messages [3], [4].

Below, we present background information using one of the most widespread applications in IoT, VANET, but our solution is not limited to VANET but also has application scenarios in health care, chemistry, etc. VANET is an IoT application deployed in transportation as shown in Fig. 1, and there are five different types of communications. To communicate between the vehicles and the server, all vehicles can use an on board unit (OBU) to transmit their messages to the service center by employing road side unit (RSU). The information generated by a large number of vehicles can be further used to illustrate the real-time traffic conditions to find the best routes [5], [6]. However, adversaries may attack VANET by spreading false traffic information, and these attacks will endanger the security of the system or even threaten users' lives. Therefore, security must be considered in IoT.

To provide security for message exchange, researchers have used validation techniques to prevent the forgery attacks, but this method can not avoid deception or forgery from the communication party. To further solve this problem, the identity authentication protocol via a signature scheme has been proposed, which can verify the identity of the communication parties. In network communications, the recipient should verify the legitimacy of the received message to avoid receiving incorrect or fake messages. In general, there are five well-established security

Manuscript received 28 September 2022; revised 5 April 2023; accepted 19 April 2023. Date of publication 8 May 2023; date of current version 12 December 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62272287 and Grant U2001205, in part by the Research Funds of Guangxi Key Laboratory of Cryptography and Information Security under Grant GCIS202108, in part by the Research Funds of Henan Key Laboratory of Network Cryptography Technology under Grant LNCT2021-A04, and in part by the Fundamental Research Funds for the Central Universities. *(Corresponding authors: Yanwei Zhou; Bo Yang.)*

Kui Ma, Chunsheng Dong, and Bo Yang are with the School of Computer Science, Shaanxi Normal University, Xi'an 710062, China (e-mail: makui@snnu.edu.cn; chunshengdong@snnu.edu.cn; byang@snnu.edu.cn).

Yanwei Zhou is with the School of Computer Science, State Key Laboratory of Cryptology, Shaanxi Normal University, Xi'an 710062, China (e-mail: zyw_snnu@foxmail.com).

Ying Wang is with the Xi'an Aeronautics Computing Technique Research Institute, Aviation Industry Corporation of China, Xi'an 710089, China (e-mail: 3452693264@qq.com).

Zhe Xia is with the School of Computer Science and Technology, Wuhan University of Technology, Wuhan 430062, China (e-mail: xiazhe@whut.edu.cn).

Mingwu Zhang is with the School of Computer Science, Hubei University Of Technology, Wuhan 430068, China (e-mail: csmwzhang@gmail.com).

Digital Object Identifier 10.1109/JSYST.2023.3269597

1937-9234 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

services for cryptography: data confidentiality, data integrity, authentication, nonrepudiation, and access control. Out of these, data confidentiality, data integrity, and authentication are provided by digital signatures. Therefore, the digital signature is widely used in the communication of IoT, i.e., the corresponding signature of the transmission message is created by employing the private key of the sender, and then the receiver uses the corresponding public key of the sender to verify the validity of the signature. And to further achieve more efficient message verification, many researchers are focusing on designing efficient signature schemes with proven security [7], [8], [9], [10], [11]. Especially, certificateless public-key cryptography (CL-PKC) does not require the management of a certificate and has no key escrow problem. Due to these advantages, the certificateless signature (CLS) scheme is widely used in the design of identity authentication protocols. However, we find that many existing constructions are either computationally inefficient or fail to achieve their claimed security. To address the above shortcomings, we design an efficient CLS scheme with proven security in this article.

A. Related Works

In identity-based signature (IBS) schemes, the private key of a user is completely generated by the key generation center (KGC). Thus, the KGC knows the private key for any user. Therefore, IBS suffers from the key escrow problem. To address the above security flaws, the definition and security model of CL-PKC were proposed by Al-Riyami et al. [12], in which the user's public key and private key both contain two parts, one produced by the user himself and one by the KGC. Hence, CL-PKC can not only avoid the certificate management problem in public key infrastructure but also solve the key escrow problem in identity-based cryptography. Due to the above advantages of CL-PKC, several concrete constructions of the CLS scheme [13], [14], [15], [16], [17] have been proposed. However, many of these CLS schemes are created based on bilinear mapping, and they are inefficient as the pairing operation is computationally expensive. For this reason, the researchers began to design secure CLS schemes without using bilinear mapping.

In the past few years, how to design a CLS scheme without bilinear mapping has become a hot topic. Gong et al. [18] have created a CLS scheme to achieve stronger security. However, Yeh et al. [19] have shown that the above CLS scheme cannot keep its claimed security. To obtain a CLS scheme with stronger security, Karati et al. [20] have designed a scheme that can be proved based on the hardness of the elliptic curve discrete logarithm problem (ECDLP). Nevertheless, in [21], Pakniat and Vanda shown that the above CLS scheme is insecure for any Type I adversary. To provide better security protection for IoT, an efficient provably secure CLS scheme was proposed by Jia et al. [22], and the unforgeability was proved. However, Du et al. [23] have shown that Jia et al.'s proposal cannot keep its original security because a Type I adversary can forge a valid signature for any message by replacing the corresponding public key of the user. To further solve this security flaw, a new CLS scheme was designed in [23]. Recently, Xiang et al. [24] also showed that Jia et al.'s scheme cannot resist Type II adversary attacks, and the above adversary can forge a valid signature for any new message by employing the master private key he/she has. Meanwhile, Xiang et al. [24] put forward an efficient and secure CLS scheme. Furthermore, for Wang et al.'s CLS scheme [25], the formal security proof was omitted, and the computational costs are high. However, we found that Du et al.'s CLS scheme and Xiang et al.'s CLS scheme are also insecure, a Type I adversary can forge a valid signature for any message by replacing the original public key of the user.

B. Our Contributions

In this article, we first analyze the security flaws of the existing CLS schemes [23], [24] and point out that these constructions can not maintain their claimed security. A Type I adversary can forge a legitimate signature by replacing the user's corresponding public key. Furthermore, we show that the main security flaw of the above constructions is that although some elements of the public key are associated with the private key, not all of them are associated. This means that the hash value of all elements of the public key is not used to generate the private key of the user. In this case, the receiver of the signature cannot detect a change of the public key, even if a Type I adversary replaces the corresponding public key of the signer. To further solve the above security flaws, we propose an efficient CLS scheme with better performance. Based on the hardness of ECDLP, we use the Forking lemma to prove the unforgeability of our proposal. Furthermore, the analyses demonstrate that our construction enjoys high computation efficiency and has stronger security. The main points of our article are as follows.

1) For Du et al.'s [23] and Xiang et al.'s [24] CLS schemes, some attacks are proposed, which shows that the above constructions are insecure for any Type I adversary. Moreover, the main reason for the above security flaws is analyzed in detail. Notice that, some other existing CLS schemes [26], [27], [28], [29] also suffer similar flaws. Hence, our analyses can help the researchers to avoid the above security flaws in future studies.
2) Based on the above analyses, we propose an efficient CLS scheme without bilinear mapping and prove its security in the random oracle model, assuming ECDLP is hard. That is, our proposal has existential unforgeability under adaptive chosen message attacks (EUF-CMA). Furthermore, the performance analysis shows that our scheme is more efficient than the existing schemes. In addition, our construction has stronger security features and it is more suitable for IoT, e.g., VANET and other network environments.
3) To illustrate an application of our CLS scheme, we design an identity authentication protocol for IoT using our basic CLS scheme. The security properties such as anonymity, mutual authentication, forward and backward security, and unlinkability are formally analyzed. In identity authentication protocol, our focus is on how to verify the legitimacy of the participant's identity. For transmission efficiency and security in the communication process between the two parties, the readers can refer to [30].

II. Preliminaries

The definitions of the symbols used in our proposal are shown in Table I.

A. Mathematics Related Concepts

Eliptic curve cryptography (ECC): Define an ECC over a finite field F_q, which is denoted as $F(F_q)$, where q is a large prime

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYMBOL DEFINITION USED IN OUR PROPOSAL

Symbol	Definition
κ	Security parameter
KGC	Key generation center
ID	Identity of user
G	Elliptic additive group
P	Generator of group
$H_{i(i=1,2,3)}$	Secure hash functions
$Params$	System public parameter
msk	Master secret key of KGC
P_{pub}	Public key of KGC
D_p	Partial private key of user
SK	Private key of user
$\mathcal{A}_1, \mathcal{A}_2$	Type I and Type II adversaries
PK	Public key of user
$\mathcal{A} \rightarrow \mathcal{B} : m$	$\mathcal{A}$ sends message m to $\mathcal{B}$
m	Communication message
$a \leftarrow_R A$	Pick a uniformly at random from set A
δ	Digital signature
$y \leftarrow B(x)$	The output of algorithm B is y when the input is x

number. $F(F_q)$ adapts this equation $y^2 = x^2 + \nu x + \xi \bmod q$, where $\nu, \xi \in F(F_q)$ and $4\nu^3 + 27\xi^2 \bmod q \neq 0$.

Set G is an additive cyclic group, m be the order of group G and P be a generator of G. We call $lP = \underbrace{P + P + \cdots + P}_{l}$ (l times, $l \in Z_q^*$) as a scalar multiplication over G.

Elliptic curve discrete logarithm problem (ECDLP): G is a cyclic group, q is a lager prime, which is the order of G. P is a generator of G. For a random value $R = rP \in G$, where $r \in Z_q^*$, it is difficult to compute r in probabilistic polynomial time (PPT). The ECDLP shows that the advantage of any PPT adversary solving the hardness of ECDLP is negligible. That is, $\Pr[\mathcal{A}(P, rP) = r] \leq \mathsf{negl}(\kappa)$, where $\mathsf{negl}(\kappa)$ is a negligible value over the security parameter κ.

B. Forking Lemma

The conclusions of [31], [32], and [33] demonstrate that for a signature scheme if there exists an adversary who can forge a valid signature $\delta = (m, \sigma, h)$ with an obvious ε, then, by replaying the random oracle H, the adversary can output a new valid signature $\delta^* = (m, \sigma^*, h^*)$ with a nonnegligible advantage $\left(1 - \frac{1}{e}\right) \frac{\varepsilon}{q_h}$ based on the same random value and different the replies of the random oracle, this means that $h \neq h^*$, where h and h^* are the different responses of the random oracle H, e is the base of natural log and q_h is the number of random oracle query. In this case, the corresponding challenger can solve the hardness of complexity assumption by employing the above two valid signatures, because the challenger has two equations for two unknown random values, one of which is the solution of the corresponding hard problem. Therefore, the correct solution for the above hard problem can be outputted by the challenger.

Especially, the description of the Forking lemma is introduced in detail in [31], [32], and [33], and will not be repeated in this article due to the consistent content.

III. CERTIFICATELESS SIGNATURE SCHEME

A. Formal Definition

Similar to a certificateless public key encryption scheme, the system environment of CLS is created by KGC, and a CLS scheme involves two entities, i.e., signer and verifier. Then, a CLS scheme has the following seven algorithms.

1) Setup: The setup algorithm is run by KGC, input a security parameter κ, and outputs a system public parameters $Params$ and a master secret key msk. This algorithm can be written as $(Params, msk) \leftarrow \mathsf{Setup}(1^\kappa)$.
2) Partial-private-key-extract: KGC runs partial private key generation algorithm with $Params$, msk, and ID as input, and outputs the partial private key D_p for the user with identity ID. This algorithm can be written as $D_p \leftarrow \mathsf{Partial\text{-}Private\text{-}Key\text{-}Extract}(Params, msk, ID)$.
3) Set-secret-value: The user runs the secret value generation algorithm with $Params$ and ID as input, outputs x as its secret value. This algorithm can be written as $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID)$.
4) Set-private-key: This algorithm takes $Params$, x and D_p as input, and the private key SK corresponding to the user with identity ID is displayed. This algorithm can be written as $SK \leftarrow \mathsf{Set\text{-}Private\text{-}Key}(Params, x, D_p)$.
5) Set-public-key: This algorithm takes $Params$, x and D_p as input, and outputs the corresponding public key PK of the user with identity ID. This algorithm can be written as $PK \leftarrow \mathsf{Set\text{-}Public\text{-}Key}(Params, x, D_p)$.
6) Sign: The signer runs the signature algorithm with $Params$, ID, m and the private key SK of signer as input, and outputs the corresponding signature σ on the message m. This algorithm can be written as $\sigma \leftarrow \mathsf{Sign}(Params, ID, m, SK)$.
7) Verify: The verifier receiving the signature message pair (σ, m), and runs the verify algorithm with $Params$, ID, σ, m and the public key PK of signer as input. If σ is a valid signature, then output "1"; Otherwise, output "0." This algorithm can be written as $0/1 \leftarrow \mathsf{Verify}(Params, ID, \sigma, m, PK)$.

For the sake of simplicity, the functions of these four algorithms Partial-Private-key-Extract, Set-Secret-Value, Set-Private-Value, and Set-Public-Value can be achieved with an interactive key generation algorithm KeyGen, which is defined as follows.

KeyGen: This is an interactive algorithm between KGC and the user. In the beginning, the user with identity ID creates a secret value x and sends its identity ID and the corresponding public parameter (which is created with secret value x) to KGC. After that, KGC produced the partial private key D_p for the user and sends it to the user. Finally, the user outputs the private key SK and the public key PK. Hence, this algorithm can be written as $(PK, SK) \leftarrow \mathsf{KeyGen}(1^\kappa)$. We have to stress that, in the above algorithm, a secret value and partial private key are also generated.

B. Security Model

CL-PKC will face two kinds of attacks, one is from outside attacks (it is a malicious user), and one is from inside attacks (it is a malicious KGC). Similarly, the CLS scheme faces two kinds of PPT adversaries, one is Type I adversary $\mathcal{A}_1$, and the other is Type II adversary $\mathcal{A}_2$, where $\mathcal{A}_1$ is a malicious user, which can replace the public key of any user but not know the master secret key, $\mathcal{A}_2$ is a malicious KGC, which has obtained the master secret key and cannot replace the public key of any user. Especially, $\mathcal{A}_1$ cannot replace the corresponding public key of the challenge identity before the challenge stage, and $\mathcal{A}_2$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

does not need to perform the partial private key generation query because he/she can calculate it himself.

We classify the adversary $\mathcal{A}_1$ according to his/her abilities, such as each adversary can be normal adversary $\mathcal{A}_1$, strong adversary $\mathcal{A}_1$ and super adversary $\mathcal{A}_1$. The normal $\mathcal{A}_1$ can replace the user's public key, however, the signature query will be answered by employing the original public key. For the strong $\mathcal{A}_1$, the challenger can obtain the secret value of replaced public key, thus, in the signature query, the challenger can generate the corresponding signature by using the replaced public key. For the super $\mathcal{A}_1$, the challenger can answer the corresponding signature query for an identity without using the corresponding secret value of the public key, even if the public key corresponding to the above identity has not been replaced. From the above discussions, it can be seen that the super $\mathcal{A}_1$ is more suitable for the application requirements of the actual application. If a CLS scheme can resist a super $\mathcal{A}_1$, then it also can resist the strong and the normal adversaries. Especially, since $\mathcal{A}_2$ cannot replace the public key of the user, the challenger can use the corresponding secret value of the public key to generate the response of the signature query submitted by $\mathcal{A}_2$, so $\mathcal{A}_2$ is not classified.

For the CLS scheme, we will define the following two security games to show the unforgeability of the CLS scheme under choose message attacks, in which, one is performed by a Type I super adversary $\mathcal{A}_1$ and a challenger $\mathcal{C}$, and the other is performed by a Type II adversary $\mathcal{A}_2$ and a challenger $\mathcal{C}$.

Game 1: This game is performed by $\mathcal{C}$ and $\mathcal{A}_1$, and the specific details are described as follows.

Initialization: $\mathcal{C}$ runs Setup to generate master secret key msk and public parameters $Params$, and sends $Params$ to $\mathcal{A}_1$. Also, $\mathcal{C}$ keeps msk as a secret.

Queries: In this stage, the following queries can be adaptively submitted by $\mathcal{A}_1$. That is, the adversary will make a new query based on the received response.

1) *Creation-user(ID_i):* When receiving a creation user query for an identity ID_i, $\mathcal{C}$ runs $D_p \leftarrow \mathsf{Partial\text{-}Private\text{-}Key\text{-}Extract}(Params, msk, ID_i)$, $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID_i)$ and $PK \leftarrow \mathsf{Set\text{-}Public\text{-}Key}(Params, x, D_p)$, and outputs PK to $\mathcal{A}_1$.
2) *Extract-partial-private-key(ID_i):* When $\mathcal{C}$ receiving this query from $\mathcal{A}_1$ with ID_i as input, $\mathcal{C}$ runs $D_p \leftarrow \mathsf{Partial\text{-}Private\text{-}Key\text{-}Extract}(Params, msk, ID_i)$, and outputs D_p to $\mathcal{A}_1$.
3) *Extract-secret-value(ID_i):* When $\mathcal{C}$ receiving this query from $\mathcal{A}_1$ with an identity ID_i as input, $\mathcal{C}$ runs $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID_i)$, and outputs x to $\mathcal{A}_1$.
4) *Extract-private-key(ID_i):* When $\mathcal{C}$ receiving this query from $\mathcal{A}_1$ with an identity ID_i as input, $\mathcal{C}$ runs $D_p \leftarrow \mathsf{Partial\text{-}Private\text{-}Key\text{-}Extract}(Params, msk, ID_i)$, at the same time, $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID_i)$ and $SK \leftarrow \mathsf{Set\text{-}Private\text{-}Key}(Params, x, D_p)$, and outputs SK to $\mathcal{A}_1$.
5) *Replace-public-key (ID_i, PK^*):* When receiving a public key replacement query for the identity value tuple (ID_i, PK^*), the corresponding public key PK of user with identity ID_i will be replaced with a new value PK^*.
6) *Sign(ID_i, m, PK_i):* When receiving this query for the identity message pair (ID_i, m), $\mathcal{C}$ runs $\sigma \leftarrow \mathsf{Sign}(Params, ID_i, m, SK)$, where SK can be obtained by running private key generation query with ID_i as input. Finally, $\mathcal{C}$ outputs σ as the corresponding signature of the message m to $\mathcal{A}_1$.

Forgery: In this stage, $\mathcal{A}_1$ outputs a forgery signature σ^* for the identity message pair (m^*, ID_i^*), $\mathcal{A}_1$ will wins in this game if the following conditions hold.

1) σ^* is valid for the identity message pair (ID_i^*, m^*).
2) The corresponding signature generation query for (ID_i^*, m^*) never be submitted by $\mathcal{A}_1$.
3) The *extract-partial-private-key* and *extract-private-key* queries for ID_i^* never be submitted by $\mathcal{A}_1$.

Therefore, the advantage of $\mathcal{A}_1$ wins in Game 1 is defined by $\mathsf{Adv}_{\mathcal{A}_1}(\kappa) = \Pr[\mathcal{A}_1\, wins]$.

Game 2: This game is performed by $\mathcal{C}$ and $\mathcal{A}_2$, and the specific details are described as follows.

Initialization: $\mathcal{C}$ runs Setup to generate master secret key msk and public parameters $Params$, and sends $Params$ and msk to $\mathcal{A}_2$.

Queries: In this stage, the following queries can be adaptively submitted by $\mathcal{A}_2$.

1) *Creation-user(ID_i):* When receiving a creation user query for an identity ID_i, $\mathcal{C}$ runs $D_p \leftarrow \mathsf{Partial\text{-}Private\text{-}Key\text{-}Extract}(Params, msk, ID_i)$, $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID_i)$ and $PK \leftarrow \mathsf{Set\text{-}Public\text{-}Key}(Params, x, D_p)$, and outputs PK to $\mathcal{A}_2$.
2) *Extract-secret-value(ID_i):* When $\mathcal{C}$ receiving this query from $\mathcal{A}_2$ with an identity ID_i as input, $\mathcal{C}$ runs $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID_i)$, and outputs x to $\mathcal{A}_2$.
3) *Extract-private-key(ID_i):* When $\mathcal{C}$ receiving this query from $\mathcal{A}_2$ with an identity ID_i as input, $\mathcal{C}$ runs $D_p \leftarrow \mathsf{Partial\text{-}Private\text{-}Key\text{-}Extract}(Params, msk, ID_i)$, $x \leftarrow \mathsf{Set\text{-}Secret\text{-}Value}(Params, ID_i)$ and $SK \leftarrow \mathsf{Set\text{-}Private\text{-}Key}(Params, x, D_p)$, after that, outputs SK to $\mathcal{A}_2$.
4) *Sign(ID_i, m, PK_i):* When $\mathcal{C}$ receiving this query for the identity message pair (ID_i, m), $\mathcal{C}$ runs $\sigma \leftarrow \mathsf{Sign}(Params, ID_i, m, SK)$, where SK can be obtained by running private key generation query with ID_i as input. Finally, $\mathcal{C}$ outputs σ as the corresponding signature of the message m to $\mathcal{A}_2$.

Forgery: In this stage, $\mathcal{A}_2$ outputs a forgery signature σ^* for the identity message pair (m^*, ID_i^*), $\mathcal{A}_2$ wins this game if the following conditions hold.

1) σ^* is valid for the identity message pair (ID_i^*, m^*).
2) The corresponding signature generation query for (ID_i^*, m^*) never be submitted by $\mathcal{A}_2$.
3) The *extract-secret-value* and *extract-private-key* queries for ID_i^* never be submitted by $\mathcal{A}_2$.

Therefore, the advantage of $\mathcal{A}_2$ wins in Game 2 is defined by $\mathsf{Adv}_{\mathcal{A}_2}(\kappa) = \Pr[\mathcal{A}_2\, wins]$.

Definition 1 (Unforgeability): For any PPT adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$, if the corresponding advantages are negligible, that is $\mathsf{Adv}_{\mathcal{A}_1}(\kappa) \leq \mathsf{negl}(\kappa)$ and $\mathsf{Adv}_{\mathcal{A}_2}(\kappa) \leq \mathsf{negl}(\kappa)$, then the corresponding CLS scheme has EUF-CMA security.

Notice that, for convenience, the super adversary $\mathcal{A}_1$ is referred to as the adversary $\mathcal{A}_1$ in the following.

IV. Security Flaws of the Previous CLS Schemes

In this section, we have analyzed in detail the security of the previous CLS schemes [23], [24], and point out that, for PPT adversary $\mathcal{A}_1$, the concrete constructions of Du et al.'s

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

scheme [23] and Xiang et al.'s scheme [24] cannot keep their claimed security, because $\mathcal{A}_1$ can forge a valid signature for any message by replacing the public key of the corresponding user. Furthermore, we will also briefly analyze the reasons for security flaws in the above CLS schemes.

A. Cryptanalysis for Du et al.'s CLS Scheme

First, Du et al.'s CLS scheme [23] will be reviewed in this subsection, which has the following seven algorithms.

1) Setup: Let G be an additive cyclic group of order prime number q over the finite field F_q, and P be a generator of G. Let $H_{i(i=1,2,3)} : \{0,1\}^* \to Z_q^*$ be three secure hash functions. KGC selects $s \in Z_q^*$, calculates $P_{pub} = sP$, set $msk = s$ as the master secret key. Finally, the system parameters are published with $ppk = \{q, G, P, P_{pub}, H_1, H_2, H_3\}$.
2) Extract-partial-private-key: For a user with identity ID, KGC chooses $\alpha \in Z_q^*$, calculates $R_{ID} = \alpha P$, $h_1 = H_1(ID, R_{ID}, P, P_{pub})$ and $d_{ID} = \alpha + h_1 s \mod q$. KGC sends $D_{ID} = (R_{ID}, d_{ID})$ to the user.
3) Set-secret-value: The user calculates $X_{ID} = x_{ID}P$, where $x_{ID} \in Z_q^*$, and set x_{ID} as the secret value.
4) Set-public-key: The user sets $PK_{ID} = (R_{ID}, X_{ID})$ as the public key.
5) Set-private-key: The user sets $SK_{ID} = (d_{ID}, x_{ID})$ as the private key.
6) Sign: A signer with identity ID randomly selects $t \in Z_q^*$, after that, computes $T = tP$, $h_2 = H_2(m, T, ID, PK_{ID}, P_{pub})$, $h_3 = H_3(m, T, PK_{ID}, h_2)$ and $z = t^{-1}(h_2 x_{ID} + h_3 d_{ID}) \mod q$. Finally, outputs $\sigma = (T, z)$ as the signature on the message m.
7) Verify: After receiving a signature $\sigma = (T, z)$, the verifier computes $h_1 = H_1(ID, R_{ID}, P, P_{pub})$, $h_2 = H_2(m, T, ID, PK_{ID}, P_{pub})$ and $h_3 = H_3(ID, m, T, PK_{ID}, h_2)$. If $zT = h_2 X_{ID} + h_3(R_{ID} + h_1 P_{pub})$ holds, then output "1"; Otherwise, output "0."

Du et al. [23] declared their CLS scheme can resist super $\mathcal{A}_1$ and $\mathcal{A}_2$. However, we find that the above scheme cannot keep its original security. For a PPT adversary $\mathcal{A}_1$, a legal signature can be forged, and the specific process is as follows.

1) For a user with identity ID, the original public key $PK_{ID} = (R_{ID}, X_{ID})$ is replaced with $PK^*_{ID} = (X^*_{ID}, R_{ID})$ by $\mathcal{A}_1$, where $X^*_{ID} = h_1 P_{pub} + R_{ID}$ and $h_1 = H_1(ID, R_{ID}, P, P_{pub})$. By this replacement, all external users think that the public key of the above user is $PK^*_{ID} = (X^*_{ID}, R_{ID})$.
2) For any message m, a forged signature $\sigma^* = (T^*, z^*)$ can be created by $\mathcal{A}_1$ through these computations.
   Compute $T^* = X^*_{ID}$ and $z^* = h^*_3 + h^*_2$, where $h^*_2 = H_2(m, T^*, ID, PK^*_{ID}, P_{pub})$ and $h^*_3 = H_3(ID, m, T^*, PK^*_{ID}, h^*_2)$.
3) The verifier receives the above forged signature $\sigma^* = (T^*, z^*)$, because $z^*T^* = h^*_2 X^*_{ID} + h^*_3(R_{ID} + h_1 P_{pub})$ holds, and output "1," that means $\sigma^* = (T^*, z^*)$ is valid. Specially, the correctness can be obtained through this equation $z^*T^* = (h^*_3 + h^*_2)X^*_{ID} = h^*_2 X^*_{ID} + h^*_3 X^*_{ID} = h^*_2 X^*_{ID} + h^*_3(R_{ID} + h_1 P_{pub})$.

As discussed above, we have that Du et al.'s CLS scheme [23] is insecure for the PPT adversary $\mathcal{A}_1$.

B. Cryptanalysis for Xiang et al.'s CLS Scheme

First, Xiang et al.'s CLS scheme [24] will be reviewed in this subsection, which has the following seven algorithms.

1) Setup: Let $H_{i(i=1,2,3)} : \{0,1\}^* \to Z_q^*$ be three secure hash functions. KGC selects $s \in Z_q^*$ and calculates $P_{pub} = sP$, set $msk = s$ as the master secret key. Finally, the system parameters is published with $ppk = \{q, G, P, P_{pub}, H_1, H_2, H_3\}$.
2) Extract-partial-private-key: For a user with identity ID, KGC chooses $r \in Z_q^*$ and calculates $R = rP$, $h_1 = H_1(ID, R, P_{pub})$ and $d = r + h_1 s \mod q$. KGC sends (R, d) to the user.
3) Set-secret-value: The user calculates $X = xP$, where $x \in Z_q^*$, and set x as the secret value.
4) Set-public-key: Set $PK = (R, X)$ as the public key.
5) Set-private-key: Set $SK = (d, x)$ as the private key.
6) Sign: A signer with identity ID randomly selects $t \in Z_q^*$, after that, computes $T = tP$, $h_2 = H_2(T, ID, PK)$, $h_3 = H_3(m, T, ID, PK, P_{pub})$ and $\tau = x^{-1}(h_2 t + h_3 d) \mod q$. Finally, output $\sigma = (T, \tau)$ as the signature on the message m.
7) Verify: After receiving a signature $\sigma = (T, z)$, the verifier first computes $h_1 = H_1(ID, R, P, P_{pub})$, $h_2 = H_2(T, ID, PK)$ and $h_3 = H_3(m, T, ID, PK, P_{pub})$.

After that, if $\tau X = h_2 T + h_3(R + h_1 P_{pub})$ is hold, then output "1"; Otherwise, output "0."

Xiang et al.'s CLS scheme [24] is also insecure, and for the Type I adversary $\mathcal{A}_1$, a legal signature can also be forged, and the specific process is as follows.

1) For a user with identity id, the original public key $PK = (R, X)$ is replaced with $PK^* = (X^*, R)$ by $\mathcal{A}_1$, where $X^* = h_1 P_{pub} + R$ and $h_1 = H_1(ID, R, P_{pub})$.
2) For any new message m, $\mathcal{A}_1$ computes $T^* = h_1 P_{pub} + R$ and $\tau^* = h^*_2 + h^*_3$, where $h^*_2 = H_2(T^*, ID, PK)$ and $h^*_3 = H_3(m, T^*, ID, PK, P_{pub})$. Finally, outputs $\sigma^* = (T^*, z^*)$ as a forged signature.
3) The verifier receives the above forged signature $\sigma^* = (T^*, \tau^*)$, because $\tau^* X^* = h^*_2 T^* + h^*_3(R + h_1 P_{pub})$ holds, and output "1," that means $\sigma^* = (T^*, \tau^*)$ is valid. Specially, the correctness can be obtained through this equation $\tau^* X^* = (h^*_2 + h^*_3)X^* = h^*_2 T^* + h^*_3(R + h_1 P_{pub})$.

As discussed above, we have that Xiang et al.'s CLS scheme [24] is also insecure for the PPT adversary $\mathcal{A}_1$.

C. Causes of Security Flaws

We believe that, for the above CLS schemes of Du et al. [23] and Xiang et al. [24], the main reasons for the security flaws are shown as follows.

1) In [23], public key $PK_{ID} = (R_{ID}, X_{ID})$ of any user with an identity ID, the elements R_{ID} and X_{ID} are independent of each other, in this case, $\mathcal{A}_1$ can replace the public key, and the corresponding authentication operation does not detect that the public key has been replaced. Furthermore, for Du et al.'s CLS scheme, the validity of the signature $\delta = (z, T)$ is obtained by checking $zT = h_2 X_{ID} + h_3(R_{ID} + h_1 P_{pub})$, where z and T are elements of the signature. Although the corresponding input of hash functions H_2 and H_3 includes public key

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

information, the above verification form provides convenience for the adversary $\mathcal{A}_1$.

2) Similarly, in [24], public key's elements R and X of any user are also independent of each other. Hence, the reasons for security flaws in Xiang et al.'s CLS scheme are similar to those of Du et al.'s CLS scheme.

In summary, the signature schemes with similar Xiang et al.'s and Du et al.'s structures are insecure. A prerequisite for designing a secure CLS scheme is to ensure that the key generated by the KGC and the user-generated key are related to each other and that the random number used for the signature and the hash function used for the signature are related in some way.

V. New CLS Scheme With Provable Security

To obtain a CLS scheme with better performance and strong security, in this section, an efficient and secure CLS scheme is proposed without bilinear mapping, and the corresponding security can be proved with the hardness of ECDLP.

A. Concrete Construction

Our CLS scheme consists of the following four algorithms.

1) Setup: Let $H_{i(i=1,2,3)} : \{0,1\}^* \rightarrow Z_q^*$ be three secure hash functions. KGC chooses $s \in Z_q^*$ and computes $P_{pub} = sP$. Finally, KGC outputs $Params = \{G, q, P, P_{pub}, H_1, H_2, H_3\}$ as the public parameter.
2) Key generation: The user with identity ID randomly chooses $x \in Z_q^*$ and computes $X = xP$. After receive (ID, X) from the user, KGC chooses $r \in Z_q^*$, and computes $R = rP$ and $d = r + sh_1 \mod q$, where $h_1 = H_1(ID, R, X)$. Also, KGC sends (R, d) as the partial private key to the user. The user can verify the correctness of (R, d) through computing $dP = R + h_1 P_{pub}$. Finally, the user sets $PK = (X, R)$ as the public key, and sets $SK = (x, d)$ as the private key.
3) Signature: The signer chooses $t \in Z_q^*$, computes $T = tP$ and $\sigma = th_3 + h_2(x + d) \bmod q$, where $h_2 = H_2(ID, m, PK, T)$ and $h_3 = H_3(ID, m, PK, T)$. Finally, outputs $\delta = (T, \sigma)$.
4) Verification: After receiving the signature $\delta = (T, \sigma)$, the verifier first computes $h_1 = H_1(ID, R, X)$, $h_2 = H_2(ID, m, PK, T)$ and $h_3 = H_3(ID, m, PK, T)$. If $\sigma P = h_3 T + h_2(X + R + h_1 P_{pub})$ holds, then the signature is valid, and output "1"; Otherwise, output "0". Notice that, the correctness of the signature $\delta = (T, \sigma)$ can be obtained from this equation $\sigma P = th_3 P + h_2(x + d)P = h_3 T + h_2(X + R + h_1 P_{pub})$.

B. Security Proof

From the following two theorems, we can say that our construction is secure because our CLS scheme could resist the Type I adversary $\mathcal{A}_1$ and the Type II adversary $\mathcal{A}_2$. The specific proof of the theorem is as follows.

Theorem 1: If there exists a Type I adversary $\mathcal{A}_1$ who can successfully forge a valid signature with obvious advantage ϵ, then we can build an algorithm $\mathcal{C}$ which can solve the hardness of ECDLP with nonnegligible advantage $\left(1 - \frac{1}{e}\right) \frac{\varepsilon_1}{e(Q_1+Q_2+1)Q_h}$, where Q_1, Q_2, and Q_h are the number of partial private key generation query, private key generation query, and hash oracle H_2 query submitted by $\mathcal{A}_1$, respectively. In addition, e is the base of the natural log.

Proof: In the beginning, $\mathcal{C}$ will obtain an instance (P, aP) from the challenger of ECDLP, where $a \in Z_q^*$. After that, the following game will be played between $\mathcal{C}$ and $\mathcal{A}_1$. The specific process is as follows.

Initialize: $\mathcal{C}$ chooses an identity ID^* as the challenge identity, and set $P_{pub} = aP$ (where $s = a$ is unknown for $\mathcal{C}$). Finally, $\mathcal{C}$ outputs $\{G, q, P, P_{pub}, H_1, H_2, H_3\}$ as the system public parameters, where H_1, H_2, and H_3 are three random oracles. In addition, $\mathcal{C}$ maintains three initially empty lists L_u, L_2, and L_3 for tracking $\mathcal{A}_1$'s queries.

Queries stage: In this stage, the following queries will be adaptively submitted.

1) *Creation-user*(ID_i). The creation user query is submitted by $\mathcal{A}_1$ with an identity ID_i as input, if $(ID_i, d, R, X, x, h_1) \in L_u$, then $\mathcal{C}$ outputs $PK_{ID_i} = (R, X)$ to $\mathcal{A}_1$; Otherwise, $\mathcal{C}$ does the following computations.

a) If $ID_i = ID^*$, $\mathcal{C}$ chooses $r^*, x^*, h_1^* \in Z_q^*$, and computes $R^* = r^* P$ and $X^* = x^* P$. Finally, $\mathcal{C}$ outputs $PK_{ID^*} = (R^*, X^*)$ as the corresponding public key to $\mathcal{A}_1$, and adds $(ID^*, \perp, R^*, X^*, x^*, h_1^*)$ into L_u.
b) Otherwise, $\mathcal{C}$ chooses $d, x, h_1 \in Z_q^*$, and computes $R = dP - h_1 P_{pub}$ and $X = xP$. Finally, $\mathcal{C}$ outputs $PK_{ID_i} = (R, X)$ as the corresponding public key to $\mathcal{A}_1$, and adds (ID_i, d, R, X, x, h_1) into L_u.

2) *H_1-query.* $\mathcal{A}_1$ sends (ID_i, R, X) to the hash oracle H_1, $\mathcal{C}$ can find a tuple (ID_i, d, R, X, x, h_1) from L_u with (ID_i, R, X) as the index, and returns h_1 to $\mathcal{A}_1$. Notice that, $\mathcal{A}_1$ has made the creation user query for ID_i before this query.

3) *H_2-query.* When receiving a hash oracle H_2 query for the tuple (ID_i, m, PK, T), if $(ID_i, m, PK, T, h_2) \in L_2$, then $\mathcal{C}$ returns h_2 to $\mathcal{A}_1$; Otherwise, $\mathcal{C}$ chooses $h_2 \in Z_q^*$ that satisfies condition $(*, *, *, *, h_2) \notin L_2$ (that means the hash function H_2 do not collide), and returns h_2 to $\mathcal{A}_1$. Also, $\mathcal{C}$ adds (ID_i, m, PK, T, h_2) into L_2.

4) *H_3-query.* When receiving a hash oracle H_3 query for the type (ID_i, m, PK, T), if $(ID_i, m, PK, T, h_3) \in L_3$, then $\mathcal{C}$ returns h_3 to $\mathcal{A}_1$; Otherwise, $\mathcal{C}$ chooses $h_3 \in Z_q^*$ that satisfies condition $(*, *, *, *, h_3) \notin L_3$, and returns h_3 to $\mathcal{A}_1$. Also, $\mathcal{C}$ adds (ID_i, m, PK, T, h_3) into L_3.

5) *Extract-partial-private-key* (ID_i)*:* When receiving this query, if $ID_i = ID^*$, then $\mathcal{C}$ aborts, and returns $\perp$; Otherwise:

a) If $(ID_i, d, R, X, x, h_1) \in L_u$, then $\mathcal{C}$ returns (d, X).
b) Otherwise, a creation user query is submitted with ID_i as the corresponding input, after that, $\mathcal{C}$ outputs (d, X).

6) *Extract-secret-value* (ID_i)*:* When receiving this query for the identity ID_i, if $(ID_i, d, R, X, x, h_1) \in L_u$, then $\mathcal{C}$ returns x to $\mathcal{A}_1$. Otherwise, a creation user query is submitted with ID_i as the corresponding input, after that, $\mathcal{C}$ outputs x to $\mathcal{A}_1$. Specially, if the corresponding public key of ID_i is replaced, then $\mathcal{C}$ outputs $\perp$ to $\mathcal{A}_1$.

7) *Extract-secret-key* (ID_i)*:* If $ID_i = ID^*$, then $\mathcal{C}$ aborts and returns $\perp$; Otherwise, does the following operation.

a) If $(ID_i, d, R, X, x, h_1) \in L_u$, then $\mathcal{C}$ returns $SK_{ID_i} = (d, x)$ to $\mathcal{A}_1$.
b) Otherwise, a creation user query is submitted with ID_i as the corresponding input, after that, $\mathcal{C}$ outputs $SK_{ID_i} = (d, x)$ to $\mathcal{A}_1$.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

8) *Public-key-replacement* (ID_i, PK'_{ID_i})*:* When receiving this query for $(ID_i, PK'_{ID_i} = (X', R'))$, if $ID_i = ID^*$, then $\mathcal{C}$ ignores this query. Otherwise, $\mathcal{C}$ does the following operations.

a) If $(ID_i, d, R, X, x, h_1) \in L_u$, then the corresponding tuple (ID_i, d, R, X, x, h_1) is replaced with $(ID_i, \perp, R', X', \perp, h_1)$.

b) Otherwise, a creation user query is submitted with the corresponding identity ID_i as the input, after that, the corresponding tuple (ID_i, d, R, X, x, h_1) is replaced with $(ID_i, \perp, R', X', \perp, h_1)$.

9) *Sign* (ID_i, m). When receiving this query for the identity message pair (ID_i, m), $\mathcal{C}$ can obtain the corresponding tuple (ID_i, d, R, X, x, h_1) from L_u with ID_i as an index, and performs the following computations.

a) If $ID_i \neq ID^*$ and $x \neq \perp$, then $\mathcal{C}$ chooses $t \in Z_q^*$, computes $T = tP$ and $\sigma = th_3 + h_2 x + d$, where $h_2 = H_2(ID_i, m, PK_{ID_i}, T)$ and $h_3 = H_3(ID_i, m, PK_{ID_i}, T)$. Finally, $\mathcal{C}$ sends $\delta = (T, \sigma)$ as the signature on the challenge tuple (ID_i, m) to $\mathcal{A}_1$.

b) Otherwise, $\mathcal{C}$ chooses $\sigma, h_2, h_3 \in Z_q^*$, and computes $T = h_3^{-1}[\sigma P - h_2(X + R + h_1 P_{pub})]$, after that, $\mathcal{C}$ sends the signature $\delta = (T, \sigma)$ to $\mathcal{A}_1$. Furthermore, two tuples $(ID_i, m, PK_{ID_i}, T, h_2)$ and $(ID_i, m, PK_{ID_i}, T, h_3)$ are added into L_2 and L_3, respectively. We have to stress that, the challenger $\mathcal{C}$ can use the above method to respond to any signature query submitted by adversary $\mathcal{A}_1$. To make it easier for the reader to understand, we use two methods to respond to the signature queries.

Forgery: After achieve all query, $\mathcal{A}_1$ outputs a forge signature $\delta_1^* = (T^*, \sigma_1^*)$ (where $T^* = t^* P$) for the identity message pair (ID_i^*, m^*). If $ID_i^* \neq ID^*$, then $\mathcal{C}$ aborts, and outputs $\perp$; Otherwise, based on Forking lemma [31] (the details in [32], [33]), $\mathcal{C}$ changes the output of hash oracle H_2, and $\mathcal{A}_1$ can generate a new forged signature $\delta_2^* = (T^*, \sigma_2^*)$ based on the same random value t^* and different response $\tilde{h}_2^*$ of hash oracle H_2. In this case, we have

$$\begin{cases} \sigma_1^* = t^* h_3^* + h_2^*(x^* + r^* + a h_1^*) \\ \sigma_2^* = t^* h_3^* + \tilde{h}_2^*(x^* + r^* + a h_1^*). \end{cases}$$

According to the above equations, $\mathcal{C}$ outputs $a = \frac{1}{h_1^*}\left(\frac{\sigma_1^* - \sigma_2^*}{h_2^* - \tilde{h}_2^*} - x^* - r^*\right)$ as the solution of ECDLP.

We use $\mathcal{E}_1$ to denote that the above game does not abort in the query stage, and we use $\mathcal{E}_2$ to denote that the above game does not abort in the forgery stage, and use $\mathcal{E}_3$ to denote that $\mathcal{A}_1$ creates two valid signatures δ_1^* and δ_2^* in the above game. Hence, we can have $\Pr[\mathcal{E}_1] = \left(1 - \frac{1}{Q_1 + Q_2 + 1}\right)^{Q_1 + Q_2}$ and $\Pr[\mathcal{E}_2] = \frac{1}{Q_1 + Q_2 + 1}$, where Q_1 and Q_2 are the number of partial private key generation and private key generation queries submitted by $\mathcal{A}_1$, respectively. Furthermore, based on Forking lemma [31], [32], [33], we can say that if $\mathcal{A}_1$ can create a valid forged signature with advantage ε_1, then the probability that δ_1^* and δ_2^* are two valid signatures is $\Pr[\mathcal{E}_3] \geq (1 - \frac{1}{e})\frac{\varepsilon_1}{Q_h}$, where Q_h is the number of hash oracle H_2 queries submitted by $\mathcal{A}_1$ and e is the base of the natural log.

Therefore, we can have

$$\begin{aligned} &\Pr[\mathcal{E}_1 \wedge \mathcal{E}_2 \wedge \mathcal{E}_3] \\ &\geq \left(1 - \frac{1}{Q_1 + Q_2 + 1}\right)^{Q_1 + Q_2} \frac{1}{Q_1 + Q_2 + 1}\left(1 - \frac{1}{e}\right)\frac{\varepsilon_1}{Q_h} \\ &\geq \frac{1}{Q_1 + Q_2 + 1}\left(1 - \frac{1}{e}\right)\frac{\varepsilon_1}{eQ_h} = \left(1 - \frac{1}{e}\right)\frac{\varepsilon_1}{e(Q_1 + Q_2 + 1)Q_h}. \end{aligned}$$

To sum up, if there exists an adversary $\mathcal{A}_1$ who can successfully forge a signature with advantage ε_1, then we can construct an algorithm $\mathcal{C}$ which can solve the hardness of ECDLP with probability $\left(1 - \frac{1}{e}\right)\frac{\varepsilon_1}{e(Q_1 + Q_2 + 1)Q_h}$. □

Theorem 2: If there exists a PPT Type II adversary $\mathcal{A}_2$ who can successfully forge a valid signature with obvious advantage ε_2, then we can build an algorithm that can solve the hardness of ECDLP with nonnegligible advantage $\left(1 - \frac{1}{e}\right)\frac{\varepsilon_2}{e(Q_2 + Q_3 + 1)Q_h}$, where Q_3 is the number of secret value generation query.

Proof: In the beginning, $\mathcal{C}$ will obtain an instance (P, aP) from the challenger of ECDLP, where $a \in Z_q^*$. After that, the following game will be played between $\mathcal{C}$ and $\mathcal{A}_2$. The specific process of the game is as follows.

Initialize: $\mathcal{C}$ chooses $s \in Z_q^*$ and computes $P_{pub} = sP$. Finally, $\mathcal{C}$ outputs $Params = \{G, q, P, P_{pub}, H_1, H_2, H_3\}$ as the system public parameters, where H_2 and H_3 are two random oracles, $H_1 : \{0,1\}^* \to Z_q^*$ is a secure hash function. Furthermore, $\mathcal{C}$ maintains three initially empty lists L_u, L_2, and L_3 for tracking $\mathcal{A}_2$'s queries.

Queries stage: In this stage, the following queries will be adaptively submitted by $\mathcal{A}_2$.

1) *Creation-User*(ID_i)*:* The creation user query is submitted by $\mathcal{A}_2$ with an identity ID_i as input, if $(ID_i, d, R, X, x) \in L_u$, then $\mathcal{C}$ outputs $PK_{ID_i} = (R, X)$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}$ does the following computation.

a) If $ID_i = ID^*$, $\mathcal{C}$ sets $X^* = aP$, after that, chooses $r \in Z_q^*$, computes $R^* = rP$ and $d^* = r + sh_1^*$, where $h_1^* = H_1(ID^*, R^*, X^*)$. Finally, $\mathcal{C}$ outputs $PK_{ID^*} = (R^*, X^*)$ to $\mathcal{A}_2$, and adds $(ID^*, d^*, R^*, X^*, \perp)$ into L_u.

b) Otherwise, $\mathcal{C}$ chooses $r, x \in Z_q^*$, computes $R = rP$, $X = xP$ and $d = r + sh_1$, where $h_1 = H_1(ID_i, R, X)$. Finally, $\mathcal{C}$ outputs $PK_{ID_i} = (R, X)$ as the corresponding public key of the user with the identity ID_i to $\mathcal{A}_2$, and adds (ID_i, d, R, X, x) into L_u.

2) *H_2-query:* When receiving a hash oracle H_2 query for the tuple (ID_i, m, PK, T), if $(ID_i, m, PK, T, h_2) \in L_2$, then $\mathcal{C}$ returns h_2 to $\mathcal{A}_2$; Otherwise, $\mathcal{C}$ chooses $h_2 \in Z_q^*$ that satisfies condition $(*, *, *, *, h_2) \notin L_2$, and returns h_2 to $\mathcal{A}_2$. ALso, $\mathcal{C}$ adds (ID_i, m, PK, T, h_2) into L_2.

3) *H_3-query:* When receiving a hash oracle H_3 query for the tuple (ID_i, m, PK, T), if $(ID_i, m, PK, T, h_3) \in L_3$, then $\mathcal{C}$ returns h_3 to $\mathcal{A}_2$; Otherwise, $\mathcal{C}$ chooses $h_3 \in Z_q^*$ that satisfies condition $(*, *, *, *, h_3) \notin L_3$, and returns h_3 to $\mathcal{A}_2$. Also, $\mathcal{C}$ adds (ID_i, m, PK, T, h_3) into L_3.

4) *Extract-secret-value* (ID_i). If $ID_i = ID^*$, then $\mathcal{C}$ aborts, and outputs $\perp$. Otherwise, $\mathcal{C}$ does the following operation.

a) If $(ID_i, d, R, X, x) \in L_u$, then $\mathcal{C}$ returns x to $\mathcal{A}_2$.

b) Otherwise, a creation user query is submitted with ID_i as the corresponding input, after that, $\mathcal{C}$ outputs x to $\mathcal{A}_2$.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

5) *Extract-secret-key* (ID_i). When receiving this query, if $ID_i = ID^*$, then $\mathcal{C}$ aborts, and returns $\perp$; otherwise, $\mathcal{C}$ does the following operation.
a) If $(ID_i, d, R, X, x) \in L_u$, then $\mathcal{C}$ returns $SK_{ID_i} = (d, x)$ to $\mathcal{A}_2$.
b) Otherwise, a creation user query is submitted with ID_i as the input, after that, $\mathcal{C}$ outputs $SK_{ID_i} = (d, x)$ to $\mathcal{A}_2$.
6) *Sign* (ID_i, m). When receiving a sign query for the identity message pair (ID_i, m), $\mathcal{C}$ can obtain the corresponding tuple (ID_i, d, R, X, x) from L_u with ID_i as an index, and performs the following computations.
a) If $ID_i \neq ID^*$, then $\mathcal{C}$ chooses $t \in Z_q^*$, computes $T = tP$ and $\sigma = th_3 + h_2(x + d)$, where h_2 and h_3 can be obtained by searching the lists L_2 and L_3 with the tuples (ID_i, m, PK_{ID_i}, T) and (ID_i, m, PK_{ID_i}, T) as the corresponding input. Finally, $\mathcal{C}$ sends $\delta = (T, \sigma)$ as the signature to $\mathcal{A}_2$.
b) Otherwise, $\mathcal{C}$ chooses $\sigma, h_2, h_3 \in Z_q^*$, and computes $T = h_3^{-1}[\sigma P - h_2(X + R + h_1 P_{pub})]$, after that, $\mathcal{C}$ sends the signature $\delta = (T, \sigma)$ to $\mathcal{A}_2$. Furthermore, two tuples $(ID_i, m, PK_{ID_i}, T, h_2)$ and $(ID_i, m, PK_{ID_i}, T, h_3)$ are added into L_2 and L_3, respectively.

Forgery: After achieve all query, $\mathcal{A}_2$ outputs a forge signature $\delta_1^* = (T^*, \sigma_1^*)$ (where $T^* = t^*P$) for the identity message pair (ID_i^*, m^*). If $ID_i^* \neq ID^*$, then $\mathcal{C}$ aborts, and outputs $\perp$; Otherwise, based on Forking lemma [31], $\mathcal{C}$ changes the output of hash oracle H_2, and $\mathcal{A}_2$ can generate a new forged signature $\delta_2^* = (T^*, \sigma_2^*)$ based on the same random number t^* and different response $\tilde{h}_2^*$ of hash oracle H_2. In this case, we have

$$\begin{cases} \sigma_1^* = t^* h_3^* + h_2^*(a + r^* + sh_1^*) \\ \sigma_2^* = t^* h_3^* + \tilde{h}_2^*(a + r^* + sh_1^*). \end{cases}$$

According the above equations, $\mathcal{C}$ outputs $a = \frac{\sigma_1^* - \sigma_2^*}{h_2^* - \tilde{h}_2^*} - r^* - sh_1^*$ as the solution of ECDLP.

We use $\mathcal{F}_1$ to denote that the above game does not abort in the query stage, use $\mathcal{F}_2$ to denote that the above game does not abort in the forgery stage, use $\mathcal{F}_3$ to denote that $\mathcal{A}_2$ creates two valid signatures δ_1^* and δ_2^* in the above game.

Hence, we have $\Pr[\mathcal{F}_1] = \left(1 - \frac{1}{Q_2+Q_3+1}\right)^{Q_2+Q_3}$ and $\Pr[\mathcal{F}_2] = \frac{1}{Q_2+Q_3+1}$, where Q_2 and Q_3 are the number of private key generation and secret value generation queries submitted by $\mathcal{A}_2$, respectively. Furthermore, if $\mathcal{A}_2$ can create a valid forged signature with probability ε_2, then, based on Forking lemma [31], we can obtain $\Pr[\mathcal{F}_3] \geq \left(1 - \frac{1}{e}\right) \frac{\varepsilon_2}{Q_h}$. Similarly, we have $\Pr[\mathcal{F}_1 \wedge \mathcal{F}_2 \wedge \mathcal{F}_3] \geq \left(1 - \frac{1}{e}\right) \frac{\varepsilon_2}{e(Q_2+Q_3+1)Q_h}$.

To sum up, if there exists an adversary $\mathcal{A}_2$ who can break the unforgeability of our CLS scheme with advantage ε_2, then, we can build an algorithm $\mathcal{C}$ which can solve the hardness of ECDLP with probability $\left(1 - \frac{1}{e}\right) \frac{\varepsilon_2}{e(Q_2+Q_3+1)Q_h}$. □

C. Performance Analysis

The security of our CLS scheme has been perfectly proven, so in this subsection, we will analyze the basic performance of our proposals, such as computational efficiency and communication efficiency. It should be noted that communication efficiency is determined by the length of the signature.

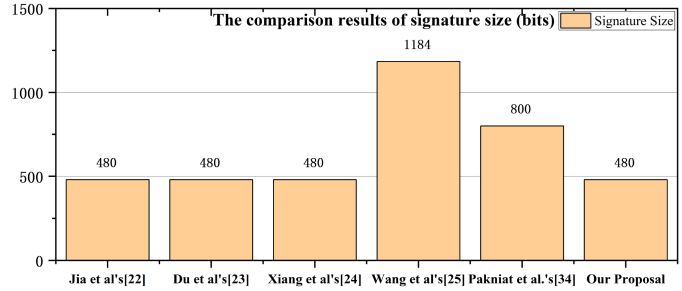

Fig. 2. Comparison results of signature size.

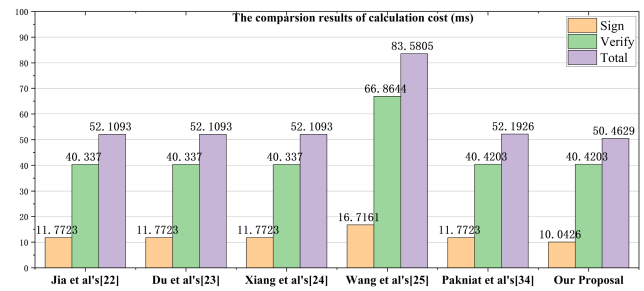

Fig. 3. Comparison results of calculation cost.

Let $|G|$ be the length of the point in the elliptic curve addition group, $|G_1|$ be the size of the point in the elliptic curve multiplicative group G_1, $|Z_q^*|$ be the length of the point in the finite field. Furthermore, the running time of the related constructs is shown in Table II, and the computation cost of a hash function is ignored. For a CLS scheme, signature and validation mainly incur a computational cost, so we only calculate the running time of the algorithm used in the above two processes as the running time of the whole CLS scheme. Since the computation cost of bilinear mapping is long, we only compare the corresponding CLS schemes using elliptic curves, and the comparison results are shown in Table II.

Notice that, the running cost of the underlying operations is shown in Table III, we use MIRACL cryptography library[1] to generate the algorithm run time in a Raspberry Pi and our hardware platform consists of a 32-b dual-core ARM Cortex-A7 and 32-b ARM Cortex-M4 with FPU/MPU processor, 1 G of memory, and runs on Linux operating system (we have published the corresponding information about the environment and data of experiment through Github[2]). To further show the comparison of computation cost and communication efficiency among different constructs, the comparison results of signature length are shown in Fig. 2, and the comparison results of running time are shown in Fig. 3.

In IoT, because mobile devices have limited power and communication range, the cost of vehicle communication is directly proportional to the size of the signature. In Table II, the signature's length of Wang et al.'s scheme [25] have $|G_1| + |Z_q^*| = 1184$ bits. In Pakniat et al.'s scheme [34], the size of signature is $2|G| + |Z_q^*| = 800$ b. The signature length of other CLS schemes is $|G| + |Z_q^*| = 480$ b. Therefore, the size

[1]https://github.com/miracl/MIRACL
[2]https://github.com/mkllsr/Algorithm-runtime-on-Raspberry-Pi

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON RESULTS OF PERFORMANCE AND SECURITY

Scheme	Signature	Validation	Signature Size	Against $\mathcal{A}_1$	Against $\mathcal{A}_2$	Protocol	Efficiency
Jia *et al.*'s [22]	$1T_{smec}+1T_{rc}$	$4T_{smec}+2T_{paec}$	$1\|G\|+1\|Z_q^*\|=480$ b	Insecure	Insecure	No	Medium
Du *et al.*'s [23]	$1T_{smec}+1T_{rc}$	$4T_{smec}+2T_{paec}$	$1\|G\|+1\|Z_q^*\|=480$ b	Insecure	$\mathcal{A}_2$	No	Medium
Xiang *et al.*'s [24]	$1T_{smec}+1T_{rc}$	$4T_{smec}+2T_{paec}$	$1\|G\|+1\|Z_q^*\|=480$ b	Insecure	$\mathcal{A}_2$	No	Medium
Wang *et al.*'s [25]	$1T_{meo}$	$4T_{meo}$	$1\|G_1\|+1\|Z_q^*\|=1184$	Super $\mathcal{A}_1$	$\mathcal{A}_2$	No	Low
Pakniat *et al.*'s [34]	$1T_{smec}+1T_{rc}$	$4T_{smec}+3T_{paec}$	$2\|G\|+1\|Z_q^*\|=800$ b	Super $\mathcal{A}_1$	$\mathcal{A}_2$	No	Medium
Our Proposal	$1T_{smec}$	$4T_{smec}+3T_{paec}$	$1\|G\|+1\|Z_q^*\|=480$ b	Super $\mathcal{A}_1$	$\mathcal{A}_2$	Yes	High

TABLE III
RUNNING COST OF THE UNDERLYING OPERATIONS

Symbol	Algorithm	Running time (ms)
T_{bpo}	Bilinear pairing operation	15.9604
T_{pbsm}	Pairing-based scalar multiplication operation	6.9452
T_{pbpa}	Pairing-based point addition operation	0.0653
T_{rc}	Modular inversion computation	1.7297
T_{meo}	Modular exponentiation operation	16.7161
T_{smec}	Scalar multiplication on eliptic curve	10.0426
T_{paec}	Point addition on eliptic curve	0.0833

of the signature of Pakniat et al.'s [34] and Wang et al.'s [25] CLS schemes are long, and the communication performance is inefficient. Our scheme has a small signature size (which is $|G|+|Z_q^*|$), and it is suitable for message transmission over VANET.

Especially, the security of the CLS scheme should be ensured. In Table II, we found that many of the previous CLS schemes took the same time to perform signature and validation operations, but they had different security performances. From Table II, we have that, our scheme can resist super $\mathcal{A}_1$ and $\mathcal{A}_2$, it has better security, however, Du et al.'s [23] and Xiang et al.'s [24] CLS schemes cannot resist Type I adversary $\mathcal{A}_1$. Also, for the adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$, Jia et al.'s [22] CLS scheme is also insecure. Despite the above CLS schemes having good communication performance, security can not be guaranteed, and it still needs to further improve.

In IoT, there are great demands on communication efficiency, so we will reduce the computation cost of cryptography primitives. From Table II, we can find that our CLS scheme is computationally efficient while resisting both types of adversary, i.e., super $\mathcal{A}_1$ and $\mathcal{A}_2$. Let's go into more detail, in our proposal, the total computational cost is $5T_{smec}+3T_{paec}=50.4629$ ms. However, the previous constructions cannot resist both type adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$, and our scheme can resist $\mathcal{A}_1$ and $\mathcal{A}_2$. This is our absolute advantage. Although Pakniat et al.'s scheme [34] solution can resist both super $\mathcal{A}_1$ and $\mathcal{A}_2$ adversaries, its performance is inferior to ours.

To sum up, we analyze the performance of our CLS scheme and other constructions and gain some results through comparison. We find that although Jia et al.'s [22], Xiang et al.'s [24] and Du et al.'s [23] constructions have high computation efficiency, but their schemes are insecure, and Pakniat et al.'s [34] scheme has a higher level of security, but looking at Figs. 2 and 3, the efficiency is not as good as ours. Hence, the comparison results show that our proposal has better performance than other solutions, and our scheme has advantages in both computational cost and security. Therefore, our solution is more suitable for VANET.

VI. ACTUAL APPLICATION OF OUR CLS SCHEME

To better validate the practicality of our scheme, we will design a communication protocol to complete the application of our signature scheme in the context of VANET. We would like to clarify here that our signature scheme is generic in the IoT domain and not only applicable to the VANET. We have chosen one of the IoT domains, the VANET, as an example to express the protocol more clearly.

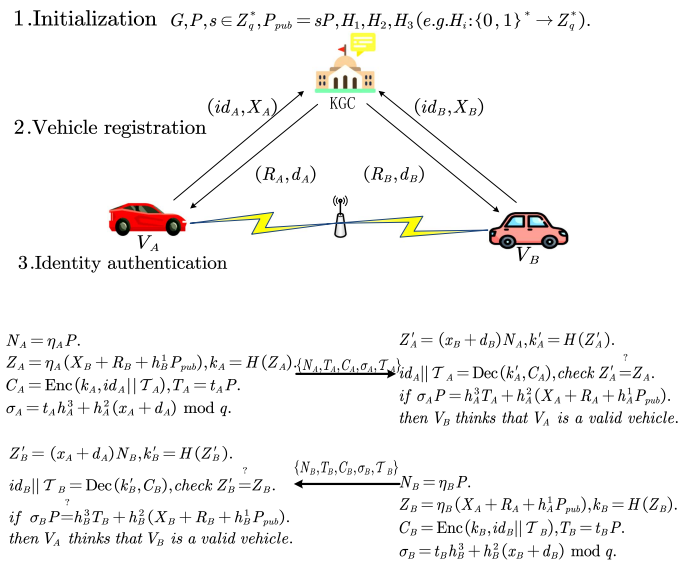

Fig. 4. Identity authentication protocol in VANET.

In VANET, dynamic information of vehicles can be effectively used through mutual information sharing to provide different functional services for vehicles. However, to ensure the effectiveness of shared messages, the communication of vehicle-to-vehicle needs to authenticate the validity of each other's identity. To achieve the above goal, in this section, we will propose an identity authentication protocol from our basic CLS scheme, which contains two participants, the sender and the receiver. As shown in Fig. 4, the two vehicles can achieve mutual identity authentication through our protocol.

In the following, we will propose the concrete construction of our identity authentication protocol, and briefly analyze its security performance.

A. Concrete Constructions

1) Initialization: At this stage, KGC generates the system parameters through the following computations.
Let $H_{i(i=1,2,3)}:\{0,1\}^*\to Z_q^*$ be three secure hash functions. Choose $s\in Z_q^*$ and compute $P_{pub}=sP$. Set $Params=\{G,q,P,P_{pub},H_1,H_2,H_3\}$ as the system public parameter.
2) Vehicle registration: At this stage, the vehicle applies for registration with KGC and generates corresponding public

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

and private key pairs (pk_i, sk_i). The public and private key pairs of any vehicle are generated through interactive communication between KGC and the vehicle.

A vehicle with identity id_i randomly chooses $x_i \in Z_q^*$, computes $X_i = x_i P$, and sends (id_i, X_i) to KGC. After that, KGC chooses $r_i \in Z_q^*$, computes $R_i = r_i P$ and $d_i = r_i + sh_1^i \mod q$, where $h_1^i = H_1(id_i, R_i, X_i)$. Finally, KGC outputs (R_i, d_i) as the partial private key.

Vehicle sets $pk_i = (X_i, R_i)$ as the public key, and sets $sk_i = (x_i, d_i)$ as the private key if $d_i P = R_i + h_1^i P_{pub}$ holds.

3) Identity authentication: At this stage, two vehicles with identities id_A and id_B achieved mutual identity validity verification through the following computations. Especially, the above two vehicles are respectively represented as V_A and V_B. Let $(\mathsf{Enc}, \mathsf{Dec})$ be semantically secure symmetric key encryption and decryption algorithms.

1) V_A generates identity authentication request messages $\{N_A, T_A, C_A, \sigma_A, \mathcal{T}_A\}$ through the following operations.
Choose $\eta_A \in Z_q^*$, and compute $N_A = \eta_A P$, $Z_A = \eta_A(X_B + R_B + h_B^1 P_{pub})$ and $C_A = \mathsf{Enc}(k_A, id_A || \mathcal{T}_A)$, where $h_B^1 = H_1(id_B, X_B, R_B)$, $k_A = H(Z_A)$ and $\mathcal{T}_A$ is the time stamp. Notice that, the encrypted transmission of identity realizes the anonymity of authentication.
Choose $t_A \in Z_q^*$, and compute $T_A = t_A P$ and $\sigma_A = t_A h_A^3 + h_A^2(x_A + d_A) \bmod q$, where $h_A^2 = H_2(id_A, C_A, PK_A, T_A)$ and $h_A^3 = H_3(id_A, C_A, PK_A, T_A)$.

2) V_B verifies the validity of V_A, and generates authentication request messages $\{N_B, T_B, C_B, \sigma_B, \mathcal{T}_B\}$ through the following operations.
Compute $Z'_A = (x_B + d_B)N_A$ and $id_A || \mathcal{T}_A = \mathsf{Dec}(k'_A, C_A)$, where $k'_A = H(Z'_A)$, and the consistency of time stamp $\mathcal{T}_A$ can avoid the replay attack, and the correctness of the above decryption operation is obtained from this equation $Z'_A = (x_B + d_B)N_A = \eta_A(x_B + d_B)P = \eta_A(X_B + R_B + h_B^1 P_{pub})$.
Compute $h_A^2 = H_2(id_A, C_A, PK_A, T_A)$ and $h_A^3 = H_3(id_A, C_A, PK_A, T_A)$. If $\sigma_A P = h_A^3 T_A + h_A^2(X_A + R_A + h_A^1 P_{pub})$ holds, then V_B thinks that V_A is a valid vehicle of VANET, where the correctness of the authentication operation is derived from this equation $\sigma_A P = t_A h_A^3 P + h_A^2(x_A + d_A)P = h_A^3 T_A + h_A^2(X_A + R_A + h_A^1 P_{pub})$.
Choose $\eta_B \in Z_q^*$ to compute $N_B = \eta_B P$, $Z_B = \eta_B(X_A + R_A + h_A^1 P_{pub})$ and $C_B = \mathsf{Enc}(k_B, id_B || \mathcal{T}_B)$, where $h_A^1 = H_1(id_A, X_A, R_A)$, $k_B = H(Z_B)$ and $\mathcal{T}_B$ is the time stamp.
Choose $t_B \in Z_q^*$, and compute $T_B = t_B P$ and $\sigma_B = t_B h_B^3 + h_B^2(x_B + d_B) \ mod \ q$, where $h_B^2 = H_2(id_B, C_B, PK_B, T_B)$ and $h_B^3 = H_3(id_B, C_B, PK_B, T_B)$.

3) V_A verifies the validity of V_B through the following operations.
Compute $Z'_B = N_B(x_A + d_A)$ and $id_B || \mathcal{T}_B = \mathsf{Dec}(k'_B, C_B)$, where $k'_B = H(Z'_B)$. Compute $h_B^2 = H_2(id_B, C_B, PK_B, T_B)$ and $h_B^3 = H_3(id_B, C_B, PK_B, T_B)$ If $\sigma_B P = h_B^3 T_B + h_B^2(X_B + R_B + h_B^1 P_{pub})$ holds, then V_A thinks that V_B is a valid vehicle of VANET.

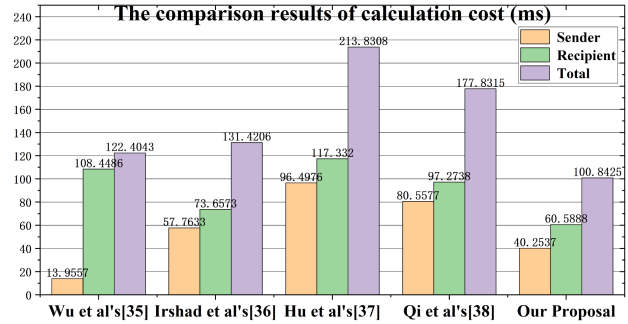

Fig. 5. Comparison results of calculation cost.

B. Security Analysis

1) Anonymity: In the above identity authentication protocol, the identity of the vehicle is encrypted, and the ciphertext can only be decrypted by the designated user (he/she can use the private key to recover the decryption key), hence, the external adversary cannot know the identity of the vehicle. More precisely, if there exists an adversary who can break the anonymity of our identity authentication protocol, then we can create an adversary who can be used to break the semantic security of the underlying symmetric encryption scheme.

2) Authentication: The authentication of our protocol is safeguarded through the unforgeability of our basic CLS scheme. We can have that the authentication messages in the above identity authentication protocol are unforgeable, that is, in our protocol, any adversary cannot forge valid request messages and response messages. In this case, if the transmissive message is validated, then it means that the corresponding sender has a legitimate identity.

3) Perfect forward and backward security: The random number and time stamps make the above identity authentication protocol have perfect forward and backward security. Also, the leakage of random numbers in one authentication process will not affect the security of the vehicles in other identity authentication stages.

4) Unlinkability: In cryptography, unlinkability refers to the property of a system or protocol that ensures that different actions or transactions carried out by the same user cannot be linked or associated with each other. In our protocol, the vehicle V_A sends a message $\{N_A, T_A, C_A, \sigma_A, \mathcal{T}_A\}$ to the vehicle V_B. During multiple interactions, there is no single adversary who can determine that the message $\{N_A, T_A, C_A, \sigma_A, \mathcal{T}_A\}$ for both interactions comes from the same vehicle. Because N_A and T_A are two random value that needs to be reselected for each interaction, hence, our protocol satisfies unlinkability. Overall, the application of unlinkability in IoV can help build trust and confidence in the use of connected vehicles and related services, while also protecting the privacy and security of users.

5) Undetectability: Undetectability typically refers to the property of a cryptographic algorithm or protocol that prevents an attacker from detecting whether a message has been encrypted or not. This means that even if an attacker intercepts a ciphertext, they cannot tell whether it is encrypted or simply random noise. In our scheme,

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON RESULTS OF PERFORMANCE

Protocol	Sender	Recipient	Total
Wu *et al.*'s [35]	$2T_{pbsm} + 1T_{pbpa}$	$2T_{bpo} + 11T_{pbsm} + 2T_{pbpa}$	$2T_{bpo} + 13T_{pbsm} + 3T_{pbpa} = 122.4043\ (ms)$
Irshad *et al.*'s [36]	$1T_{bpo} + 6T_{pbsm} + 2T_{pbpa}$	$2T_{bpo} + 6T_{pbsm} + T_{pbpa}$	$3T_{bpo} + 12T_{pbsm} + 3T_{pbpa} = 131.4206\ (ms)$
Hu *et al.*'s [37]	$7T_{pbsm} + 3T_{bpo}$	$10T_{pbsm} + 3T_{bpo}$	$13T_{pbsm} + 6T_{bpo} = 213.8308\ (ms)$
Qi *et al.*'s [38]	$4T_{bpo} + T_{meo}$	$4T_{bpo} + 2T_{meo}$	$8T_{bpo} + 3T_{meo} = 177.8315\ (ms)$
Our Proposal	$4T_{smec} + 1T_{paec}$	$6T_{smec} + 4T_{paec}$	$10T_{smec} + 5T_{paec} = 100.8425\ (ms)$

since all elements are calculated by random numbers, the encrypted message is also randomly distributed and the adversary cannot tell whether the message is encrypted or not.

6) Unobservability: Unobservability, on the other hand, typically refers to the property of a cryptographic protocol that hides the fact that a communication is taking place at all. This means that an attacker cannot even detect the existence of communication between two parties, let alone intercept and analyze any messages. In our scenario, the communication process does not leak any information about the message and the adversary cannot be bothered to determine whether or not the two parties have communicated.
7) Data integrity: In the security analysis of an encryption protocol, data integrity is an important consideration. Data integrity refers to the assurance that the data have not been modified or tampered with during transmission or storage. In our protocol, messages are signed using the designed CLS scheme, and the security of our CLS scheme is proved, which guarantees data integrity.

C. Performance Analysis

To demonstrate the efficiency of our identity authentication protocol, we briefly compare the calculation times of our protocol with other identity authentication protocols. Especially, based on the running cost of the underlying operations shown in Table II, the comparison results of the computation costs of each participant in the identity authentication protocol are shown in Table IV and Fig. 5. Notice that, for a more intuitive view of the performance gap, we only compute the computational costs of the authentication process for one interaction.

We can see that, compared with our proposal, the previous protocols [35], [36], [37], [38] consume more time in the authentication process. This is because our proposal was designed based on elliptic curves, and the other two protocols were designed based on the bilinear mapping. In summary, an identity authentication protocol designed based on our basic CLS scheme is not only more efficient but also has better security. Therefore, our protocol is more suitable for use in IoT, VANET, and other network environments.

VII. CONCLUSION

To achieve security in identity authentication, signature schemes are normally employed as the underlying technology to ensure the unforgeability of authentication messages. Since CL-PKC does not require certificate management and does not have a key escrow problem, it has been widely studied by researchers, and several concrete constructions of the CLS scheme have been proposed in the past few years. In this article, we demonstrate that some previous CLS schemes [23], [24] are insecure, and the Type I adversary can forge a valid signature for any message by replacing the public key of any user. To address these issues, we propose a new concrete proposal without the use of bilinear mapping, and the unforgeability of our CLS scheme is proved based on the hardness of ECDLP in the random oracle model. Compared with the existing CLS schemes, our proposal has higher computation efficiency and stronger security. In addition, to further provide security for message transmission of IoT, an identity authentication protocol is proposed based on our basic CLS scheme.

However, we have to note that in real-world applications, an adversary can obtain a certain amount of leakage on the secret key, and traditional security will not be maintained in the leakage setting. To further address this issue, leakage-resilient CLS schemes will be studied in our next stage based on our works [39], [40], which can provide leakage resilience for the identity authentication protocol. Another issue that will be discussed and addressed later in the work is a multifactor authentication system to improve efficiency [41].

REFERENCES

[1] D. Wang, D. Chen, B. Song, N. Guizani, X. Yu, and X. Du, "From IoT to 5G I-IoT: The next generation IoT-based intelligent algorithms and 5G technologies," *IEEE Commun. Mag.*, vol. 56, no. 10, pp. 114–120, Oct. 2018.
[2] H. HaddadPajouh, A. Dehghantanha, R. M. Parizi, M. Aledhari, and H. Karimipour, "A survey on Internet of Things security: Requirements, challenges, and solutions," *Internet Things*, vol. 14, 2021, Art. no. 100129.
[3] L. Tawalbeh et al., "IoT privacy and security: Challenges and solutions," *Appl. Sci.*, vol. 10, no. 12, 2020, Art. no. 4102.
[4] S. Balaji, K. Nathani, and R. Santhakumar, "IoT technology, applications and challenges: A contemporary survey," *Wireless Pers. Commun.*, vol. 108, no. 1, pp. 363–388, 2019.
[5] H. Zhou, W. Xu, J. Chen, and W. Wang, "Evolutionary V2X technologies toward the internet of vehicles: Challenges and opportunities," *Proc. IEEE*, vol. 108, no. 2, pp. 308–323, Feb. 2020.
[6] S. Sharma and B. Kaushik, "A survey on internet of vehicles: Applications, security issues & solutions," *Veh. Commun.*, vol. 20, 2019, Art. no. 100182.
[7] Y. Zhang, R. H. Deng, D. Zheng, J. Li, P. Wu, and J. Cao, "Efficient and robust certificateless signature for data crowdsensing in cloud-assisted industrial IoT," *IEEE Trans. Ind. Inform.*, vol. 15, no. 9, pp. 5099–5108, Sep. 2019.
[8] D. He, B. Huang, and J. Chen, "New certificateless short signature scheme," *IET Inf. Secur.*, vol. 7, no. 2, pp. 113–117, 2013.
[9] J.-L. Tsai, "A new efficient certificateless short signature scheme using bilinear pairings," *IEEE Syst. J.*, vol. 11, no. 4, pp. 2395–2402, Dec. 2017.
[10] P. Yi et al., "An efficient identity-based signature scheme with provable security," *Inf. Sci.*, vol. 576, pp. 790–799, 2021.
[11] Y. Hou, H. Xiong, X. Huang, and S. Kumari, "Certificate-based parallel key-insulated aggregate signature against fully chosen key attacks for industrial Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8935–8948, Jun. 2021.
[12] S. Sattam Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. 9th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2003, pp. 452–473.
[13] X. Yang, X. Pei, G. Chen, T. Li, M. Wang, and C. Wang, "A strongly unforgeable certificateless signature scheme and its application in IoT environments," *Sensors*, vol. 19, no. 12, 2019, Art. no. 2692.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.

[14] A. Karati, S. K. H. Islam, and M. Karuppiah, "Provably secure and lightweight certificateless signature scheme for IIoT environments," *IEEE Trans. Ind. Inform.*, vol. 14, no. 8, pp. 3701–3711, Aug. 2018.

[15] F. Rezaeibagha, Y. Mu, X. Huang, W. Yang, and K. Huang, "Fully secure lightweight certificateless signature scheme for IIoT," *IEEE Access*, vol. 7, pp. 144433–144443, 2019.

[16] L. Zhou, C. Su, and K.-H. Yeh, "A lightweight cryptographic protocol with certificateless signature for the Internet of Things," *ACM Trans. Embedded Comput. Syst.*, vol. 18, no. 3, pp. 28:1–28:10, 2019.

[17] K.-A. Shim, "A new certificateless signature scheme provably secure in the standard model," *IEEE Syst. J.*, vol. 13, no. 2, pp. 1421–1430, Jun. 2019.

[18] P. Gong and P. Li, "Further improvement of a certificateless signature scheme without pairing," *Int. J. Commun. Syst.*, vol. 27, no. 10, pp. 2083–2091, 2014.

[19] K.-H. Yeh, K.-Y. Tsai, and C.-Y. Fan, "An efficient certificateless signature scheme without bilinear pairings," *Multimedia Tools Appl.*, vol. 74, no. 16, pp. 6519–6530, 2015.

[20] A. Karati, S. K. H. Islam, and G. P. Biswas, "A pairing-free and provably secure certificateless signature scheme," *Inf. Sci.*, vol. 450, pp. 378–391, 2018.

[21] N. Pakniat and B. Abasi Vanda, "Cryptanalysis and improvement of a pairing-free certificateless signature scheme," in *Proc. 15th Int. ISC (Iranian Soc. Cryptol.) Conf. Inf. Secur. Cryptol.*, Tehran, Iran, 2018, pp. 1–5.

[22] X. Jia, D. He, Q. Liu, and K.-K. Raymond Choo, "An efficient provably-secure certificateless signature scheme for Internet-of-Things deployment," *Ad Hoc Netw.*, vol. 71, pp. 78–87, 2018.

[23] H. Du, Q. Wen, S. Zhang, and M. Gao, "A new provably secure certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 100, 2020, Art. no. 102074.

[24] D. Xiang, X. Li, J. Gao, and X. Zhang, "A secure and efficient certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 124, 2022, Art. no. 102702.

[25] L. Wang, K. Chen, Y. Long, and H. Wang, "An efficient pairing-free certificateless signature scheme for resource-limited systems," *Sci. China Inf. Sci.*, vol. 60, no. 11, pp. 1–3, 2017.

[26] J. Liu, L. Wang, and Y. Yu, "Improved security of a pairing-free certificateless aggregate signature in healthcare wireless medical sensor networks," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5256–5266, Jun. 2020.

[27] Y. Zhan, B. Wang, and R. Lu, "Cryptanalysis and improvement of a pairing-free certificateless aggregate signature in healthcare wireless medical sensor networks," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5973–5984, Apr. 2021.

[28] Y. Chen and J. Chen, "CPP-CLAS: Efficient and conditional privacy-preserving certificateless aggregate signature scheme for VANETs," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 10354–10365, Jun. 2022.

[29] H. Yu and R. Ren, "Certificateless elliptic curve aggregate signcryption scheme," *IEEE Syst. J.*, vol. 16, no. 2, pp. 2347–2354, Jun. 2022.

[30] B. Han, S. Peng, C. Wu, X. Wang, and B. Wang, "LoRa-based physical layer key generation for secure V2V/V2I communications," *Sensors*, vol. 20, no. 3, 2020, Art. no. 682.

[31] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptol.*, vol. 13, no. 3, pp. 361–396, 2000.

[32] Z. Qiao, Q. Yang, Y. Zhou, and M. Zhang, "Improved secure transaction scheme with certificateless cryptographic primitives for IoT-based mobile payments," *IEEE Syst. J.*, vol. 16, no. 2, pp. 1842–1850, Jun. 2022.

[33] Z. Qiao, Y. Zhou, B. Yang, M. Zhang, T. Wang, and Z. Xia, "Secure and efficient certificate-based proxy signature schemes for industrial Internet of Things," *IEEE Syst. J.*, vol. 16, no. 3, pp. 4719–4730, Sep. 2022.

[34] N. Pakniat and B. Abasi Vanda, "Cryptanalysis and improvement of a pairing-free certificateless signature scheme," in *Proc. 15th Int. ISC (Iranian Soc. Cryptol.) Conf. Inf. Secur. Cryptol.*, 2018, pp. 1–5.

[35] T.-Y. Wu, Y.-Q. Lee, C.-M. Chen, Y. Tian, and N. A. Al-Nabhan, "An enhanced pairing-based authentication scheme for smart grid communications," *J. Ambient Intell. Humanized Comput.*, pp. 1–13, 2021, doi: 10.1007/s12652-020-02740-2.

[36] A. Irshad et al., "An improved multi-server authentication scheme for distributed mobile cloud computing services," *KSII Trans. Internet Inf. Syst.*, vol. 10, no. 12, pp. 5529–5552, 2016.

[37] B. Hu, W. Tang, and Q. Xie, "A two-factor security authentication scheme for wireless sensor networks in IoT environments," *Neurocomputing*, vol. 500, pp. 741–749, 2022.

[38] J. Qi, T. Gao, X. Deng, and C. Zhao, "A pseudonym-based certificateless privacy-preserving authentication scheme for VANETs," *Veh. Commun.*, vol. 38, 2022, Art. no. 100535.

[39] Y. Zhou, Y. Xu, Z. Qiao, B. Yang, and M. Zhang, "Continuous leakage-resilient certificate-based signcryption scheme and application in cloud computing," *Theor. Comput. Sci.*, vol. 860, pp. 1–22, 2021.

[40] Y. Zhou, B. Yang, Z. Xia, M. Zhang, and Y. Mu, "Novel generic construction of leakage-resilient PKE scheme with CCA security," *Des. Codes Cryptography*, vol. 89, no. 7, pp. 1575–1614, 2021.

[41] Q. Wang, D. Wang, C. Cheng, and D. He, "Quantum2FA: Efficient quantum-resistant two-factor authentication scheme for mobile devices," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 193–208, Jan./Feb. 2023.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 18,2026 at 09:16:02 UTC from IEEE Xplore. Restrictions apply.